

Linked-List Allocation

1 Basic Idea

Linked-List Allocation

In linked-list allocation, a file is stored as a chain of disk blocks. Each block contains a pointer to the next block of the same file, so the file blocks do not have to be contiguous on disk.

- A file is divided into disk blocks.
- The blocks may be located anywhere on the disk.
- The first word of each block stores a pointer to the next block.
- The remaining part of the block stores file data.
- The last block contains a special end marker instead of a pointer to another block.
- The directory entry only needs to store the address of the first block.

2 Linked List of Disk Blocks

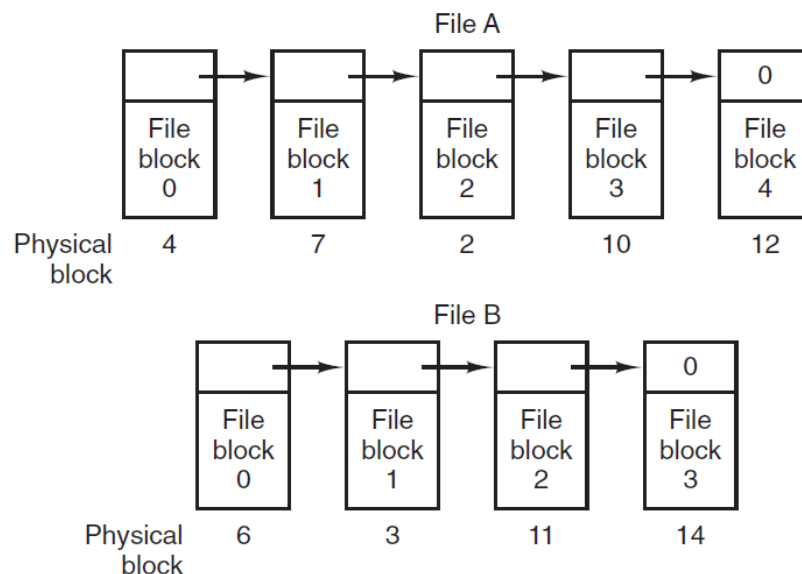


Figure 4-11. Storing a file as a linked list of disk blocks.

Figure 1: Storing files as linked lists of disk blocks.

- File A uses physical blocks $4 \rightarrow 7 \rightarrow 2 \rightarrow 10 \rightarrow 12$.
- File B uses physical blocks $6 \rightarrow 3 \rightarrow 11 \rightarrow 14$.
- The physical blocks of a file do not need to be next to each other.
- The chain is followed from the first block until the end marker is found.

3 Advantages

Advantage	Explanation
No external fragmentation	Any free disk block can be used for any file.
Simple directory entry	The directory only stores the first block number.
Files can grow easily	A new block can be linked onto the end of the file.
Good sequential access	Reading from beginning to end follows the block chain naturally.

4 Disadvantages

Slow Random Access

Random access is very slow because the system must follow the chain from the beginning to find a later block.

- To read block n , the operating system must first read blocks 1 through $n - 1$.
- This makes direct access to the middle of a file expensive.
- Each block loses a few bytes because the pointer is stored inside the block.
- The usable data area is no longer a clean power-of-two size.
- If programs expect full block-sized reads and writes, the system may need extra copying and concatenation.

5 Linked-List Allocation Using a Table

File Allocation Table

A **File Allocation Table**, or **FAT**, stores the next-block pointers in a memory table instead of storing them inside the disk blocks.

- The FAT has one entry for each physical disk block.
- Each entry tells which block comes next in the file.
- A special value such as -1 marks the end of a file.
- Unused blocks have no active chain entry.
- The directory still needs only the starting block of the file.

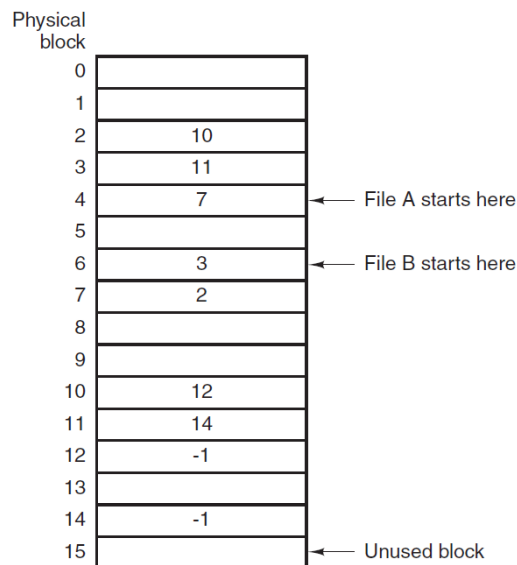


Figure 4-12. Linked-list allocation using a file-allocation table in main memory.

Figure 2: Linked-list allocation using a file-allocation table in memory.

6 FAT Example

File	Chain from FAT
File A	4 → 7 → 2 → 10 → 12 → -1
File B	6 → 3 → 11 → 14 → -1

- To locate File A, the directory gives starting block 4.
- FAT entry 4 points to 7.
- FAT entry 7 points to 2.
- The chain continues until entry 12 gives -1.
- The same idea locates File B starting from block 6.

7 Why FAT Improves the Basic Method

FAT Improvement

FAT removes the pointer from the disk block, so the entire block can store data. It also makes chain traversal faster because the chain can be followed in memory.

- Random access still requires following the chain.
- However, following the chain in memory is much faster than reading each pointer from disk.
- Full disk blocks remain available for file data.
- The file system still needs only the first block number in the directory entry.

8 Main FAT Limitation

Memory Cost

The whole FAT must be kept in memory for efficient operation. This becomes impractical on very large disks.

- A 1-TB disk with 1-KB blocks has about 1 billion blocks.
- The FAT therefore needs about 1 billion table entries.
- If each entry uses 3 bytes, the table uses about 3 GB of memory.
- If each entry uses 4 bytes for faster lookup, it uses about 4 GB of memory.
- This memory requirement shows why FAT does not scale well to large disks.

9 Comparison

Point	Basic Linked List	FAT-Based Linked List
Pointer location	First word of each disk block	Separate table in memory
Data space per block	Reduced by pointer size	Whole block available for data
Sequential access	Good	Good
Random access	Very slow, requires disk reads along the chain	Faster, chain is followed in memory
Directory entry	Stores first block	Stores first block
Main weakness	Poor random access and awkward block size	Large memory requirement

Final Point

Linked-list allocation avoids external fragmentation and allows files to grow easily, but basic linked lists make random access slow. FAT improves access by moving pointers into memory, but the table can become too large for modern large disks.